Wave Forecaster: o que já foi portado para a GPU

Relatório para o Prof. Pedro Cruz (GTA, DEL/Poli/UFRJ), em versão fácil

Matheus Hoffmann, Escola Politécnica da UFRJ
16 de outubro de 2026

> **Esta é uma versão fácil, em português, do estado do projeto ww3-gpu.**
>
> - **Vale o original.** Se este texto e os arquivos do repositório discordarem, valem os arquivos. Os principais são `kokkos/PORT_STATUS.md`, `kokkos/ports.tsv` e o `README.pt-BR.md`.
> - **Feita a partir de:** o repositório em 16 de outubro de 2026, commit `b05b4ca` de `main`.
> - **Para quem é:** o Prof. Pedro Cruz, do GTA (Grupo de Teleinformática e Automação, DEL/UFRJ), e qualquer pessoa que programa, mas não é da oceanografia nem da física.
> - **O que ficou de fora:** as equações, os detalhes de cada rotina e as tabelas completas. Cada parte encurtada aponta para o arquivo de onde veio.
> - **Escrita por:** Claude, no estilo da skill `easy-pt-br` de h0ffmann/closed-source; revisão de Matheus Hoffmann.

Os links, num lugar só

O quê	Onde
Código (GitHub)	https://github.com/h0ffmann/ww3-gpu
Site	https://h0ffmann.github.io/ww3-gpu/ (em português: https://h0ffmann.github.io/ww3-gpu/pt-BR/)
Esta página no site	https://h0ffmann.github.io/ww3-gpu/relatorio-gta/
Zenodo (DOI que sempre abre a versão mais nova)	10.5281/zenodo.23221351
Métricas do port	`kokkos/PORT_STATUS.md`, `kokkos/PORT_TABLE.tsv`, `kokkos/PORT_RANKING.md`, e a seção "port" do site
Decisões (ADRs)	`docs/ADRs/` e a página https://h0ffmann.github.io/ww3-gpu/adrs/
Desenhos de mudança (WFIPs)	`docs/WFIPs/`
Diário do que foi tentado	`docs/log/`
O livro (17 lições)	https://h0ffmann.github.io/ww3-gpu/book/ e o PDF

O quê	Onde
O pedido de acesso à H100	h0ffmann/ww3-gpu#143
Glossário de siglas	`docs/GLOSSARY.md`

Como ler as marcas

Cada afirmação deste repositório carrega uma marca. **(v)** quer dizer "verificado": alguém conferiu numa fonte ou rodou um comando, e o texto diz qual. ⚠ quer dizer "não verificado ainda". Todo número vem com o comando que o reproduz. Este relatório segue a mesma regra.

---

1. O problema, em linguagem de computação

O modelo. O WAVEWATCH III® (WW3) é o modelo de previsão de ondas do mar da NOAA, a agência americana de oceano e atmosfera. É um programa em Fortran com cerca de 263 mil linhas (v, `kokkos/ports.tsv`, medido no commit `03adda91` do WW3).

O que ele calcula. O mar é dividido numa grade de pontos. Em cada ponto, o WW3 guarda um vetor de números: quanta energia de onda existe em cada frequência e em cada direção. Esse vetor se chama espectro. A cada passo de tempo, o programa faz duas coisas:

1. **Propagação:** move a energia de um ponto da grade para os vizinhos. É parecido com um *stencil* numa imagem.
2. **Termos-fonte:** em cada ponto, sozinho, soma o que o vento coloca, o que a quebra das ondas tira e o que as ondas trocam entre si. Um ponto não depende do outro nesta etapa.

Por que a GPU ajuda. A segunda etapa é um laço sobre milhares de pontos independentes. Isso é o formato ideal para uma GPU, um *map* sobre um vetor grande. Medimos onde o tempo vai: os termos-fonte somam a maior parte do tempo de uma rodada (seção 3).

A regra que torna o problema difícil. A versão na GPU tem que dar **o mesmo resultado, bit a bit**, que o Fortran original, dentro do mesmo executável do WW3. Não basta "parecido". Se um único bit muda, a tradução está errada até que se prove o contrário (regra do `AGENTS.md`, "Bit for bit with the Fortran").

---

2. Como o código é organizado: as formas de rodar

O código do WW3 não é editado aqui. Ele vem de fora, em submódulos do git (repositórios dentro do repositório). O ww3-gpu só acrescenta a reescrita em C++ e as ferramentas em volta.

Os forks

Diretório	O que é	Para que serve
`WW3-noaa/`	o WW3 puro da NOAA, NOAA-EMC/WW3 no commit `03adda91`, sem nenhuma mudança	a referência: o resultado que precisamos reproduzir
`WW3/`	o fork h0ffmann/WW3, branch `ww3-gpu/kokkos-st4`: o WW3 puro mais um patch pequeno, 5 arquivos em `model/src/` e 813 linhas acrescentadas (v, `README.md`, lido em 2026-10-16)	colocar dentro do WW3 a chamada para o código novo
`WW4/`	o fork h0ffmann/WW4, branch `develop`: o WAVEWATCH IV, que a NOAA reescreve do zero em C++	acompanhar o sucessor do WW3
`picasso/`	h0ffmann/picasso (10.5281/zenodo.23311208): o conjunto de compiladores fixado com Nix	garantir que todos compilam com as mesmas versões

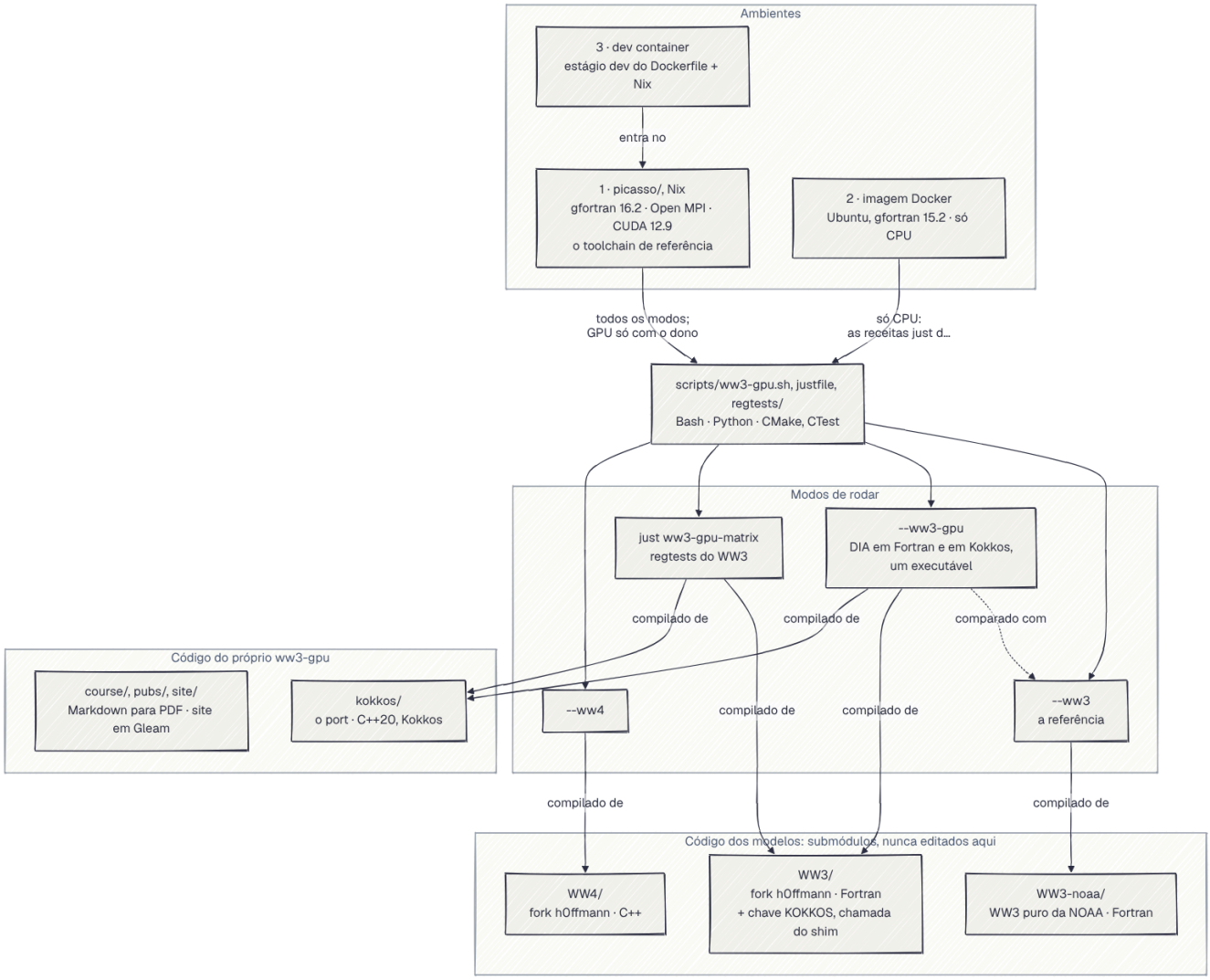
Por que existe o fork do WW3. A chamada para o código em C++ precisa estar dentro do WW3. Como este repositório nunca edita o WW3, o fork carrega essa chamada. Ela fica atrás de uma chave de compilação, `KOKKOS`. Antes de cada rodada, um script compara o fork com o WW3 puro. Se o fork mudou qualquer coisa fora do patch, o script para com erro (código de saída 3).

Os três modos de rodar

Tudo passa por um script só, `scripts/ww3-gpu.sh`:

Modo	O que roda	Compilado de
`--ww3`	o WW3 como a NOAA escreve	`WW3-noaa/`
`--ww3-gpu`	o WW3 do fork, com as rotinas portadas em Kokkos; roda o caso duas vezes no mesmo executável, uma com o Fortran e outra com o Kokkos, e compara as saídas	`WW3/` e `kokkos/`
`--ww4`	o WAVEWATCH IV; hoje ele para com código 5, porque o WW4 ainda não tem os termos-fonte nem a propagação	`WW4/`

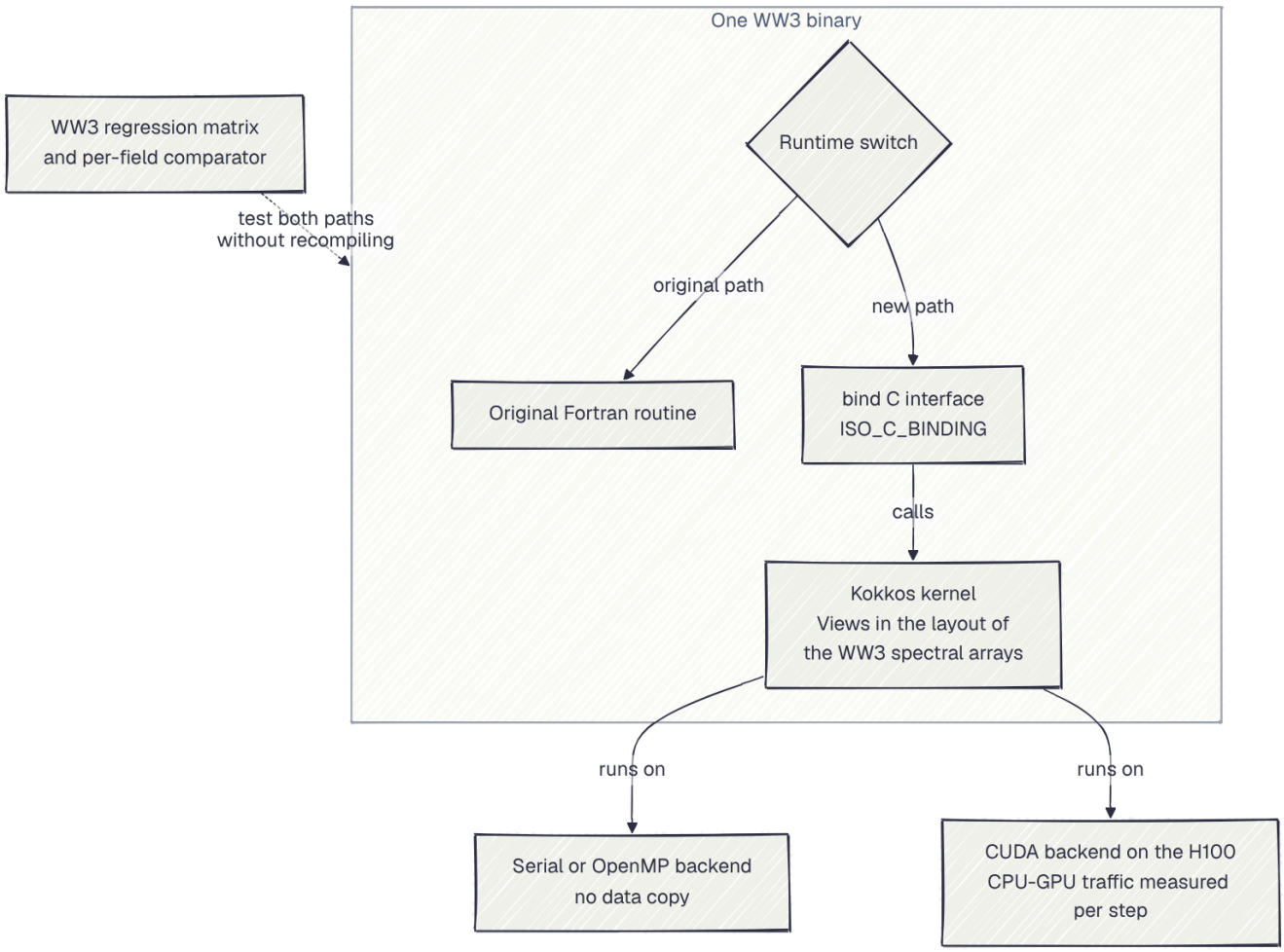
Cada modo aceita `--cpu` (o padrão) ou `--gpu`. Só o `--ww3-gpu` tem caminho de GPU. Rodar na GPU exige uma pessoa supervisionando: o script pede um "yes" digitado no terminal e para sem ele.



O sistema inteiro numa página: de onde vem o código, o que o ww3-gpu acrescenta, os modos de rodar e os ambientes.

A chave em tempo de execução

Dentro do executável do `--ww3-gpu` existem as duas versões de cada rotina portada. Uma variável de ambiente escolhe qual roda, sem recompilar: `WW_KOKKOS_SNL1=0` usa o Fortran e `WW_KOKKOS_SNL1=1` usa o Kokkos. O mesmo vale para `WW_KOKKOS_SDS4`, `WW_KOKKOS_SIN4` e `WW_KOKKOS_SPR4`. Assim o teste compara as duas versões com o mesmo binário, a mesma entrada e o mesmo compilador.



Um executável do WW3, dois caminhos: a chave escolhe a rotina original em Fortran ou o kernel em Kokkos, que roda em CPU ou em GPU.

Kokkos é uma biblioteca C++ liderada pelo Sandia National Laboratories (⚠, `docs/GLOSSARY.md`). Você escreve o kernel uma vez e compila para vários *backends*: Serial, OpenMP (vários núcleos da CPU) ou CUDA (GPU NVIDIA). A chamada do Fortran para o C++ passa por uma interface `bind(C)`, o padrão `ISO_C_BINDING` do Fortran.

Os três ambientes

O mesmo script roda em três lugares (v, `README.pt-BR.md`, "três ambientes"):

1. **picasso (Nix):** a referência. Os veredictos bit a bit e todo tempo medido saem daqui.
2. **Imagem Docker:** só CPU, só os casos pequenos, para quem não quer instalar Nix.
3. **Dev container:** para o VS Code; por dentro, ele usa o picasso.

---

3. O que já foi portado

O site e o [`PORT_STATUS.md`] contam o port em três níveis. Cada nível é mais exigente que o anterior:

Nível	O que quer dizer	Parte do tempo de rodada	Rotinas
traduzido	existe o código em Kokkos	**99,3 %**	59 de 1 023
ligado (*ported*)	o WW3 do fork chama esse código, atrás da sua chave	**87,4 %**	10 de 1 023
regtestado	os testes de regressão do próprio WW3 rodaram com ele e deram o resultado do Fortran	**48,4 %**	2 de 1 023

v. `python3 scripts/port_score.py --check`, rodado em 2026-10-16: as tabelas estão em dia. O código do port tem 16 191 linhas de C++ em 75 arquivos.

Como ler essa tabela. "Traduzido" é barato de afirmar e caro de provar. Uma rotina só sobe para "regtestado" quando roda dentro do WW3, nos casos de teste da NOAA, e dá os mesmos bytes que o Fortran. A distância entre 99,3 % e 48,4 % é exatamente o trabalho que falta.

A primeira rotina completa: `W3SNL1`

`W3SNL1` calcula as interações entre ondas (o método DIA). É a rotina mais cara: **48,4 % do tempo** de uma rodada, sozinha (v, perfil `gprof` do caso `bench_small`, 9 520 pontos de mar, 12 horas simuladas, 1 thread, 2026-10-14, `just profile-case full`).

- **Resultado:** idêntica byte a byte ao Fortran nas 100 entradas seriais da matriz de testes de regressão do WW3, e nas 25 entradas com OpenMP (v, 2026-10-15, `just ww3-gpu-matrix`, `port_report/W3SNL1_KOKKOS.md`).
- **Velocidade do kernel, fora do WW3** (1 000 pontos por chamada, i9-14900 e RTX 4090, mediana de três rodadas, 2026-09-15):

Backend	Tempo por chamada	Ganho sobre 1 thread
Serial, 1 thread	25,88 ms	1,0×
OpenMP, 32 threads	5,95 ms	4,4×
CUDA, RTX 4090	0,75 ms	34×
CUDA, só o kernel, sem cópia	0,047 ms	

Os números vêm de `bench/results/snl1-i9-14900-rtx4090-2026-09-15.json`.

Duas ressalvas honestas. Primeira: na GPU, 94 % do tempo da chamada é cópia de memória entre CPU e GPU. Hoje cada chamada copia o espectro de ida e volta (a "fase 1"). A "fase 2" deixa os dados morando na GPU, e só então o ganho do kernel aparece no modelo. Segunda: dentro do WW3, com 1 thread de CPU, a versão Kokkos ainda é **mais lenta** que o Fortran (217,9 s contra 152,7 s no `bench_small`, `just full-ww3-gpu`, 2026-10-14). O WW3 ainda não rodou com o Kokkos na GPU.

As outras rotinas

- **Física ST4** (`W3SDS4`, `W3SIN4`, `W3SPR4`: o vento, a quebra das ondas e os parâmetros médios, mais 39,0 % do tempo no mesmo perfil): ligadas ao WW3 e idênticas byte a byte nos dois casos de teste (v, 2026-10-15, `docs/log/2026-10-15-st4-checkpoint.md`). Ainda falta a matriz completa de testes para chamá-las de "regtestadas".
- **`W3SRCE`** (a rotina que chama todos os termos-fonte): traduzida e ligada num branch separado do fork; espera o seu ponto de checagem.
- **Propagação e o resto do laço de tempo** (`W3XYP3`, `W3QCK1/2/3`, `W3GATH`, `W3SCAT` e outras): traduzidas em 2026-10-16, com a paridade conferida só lendo o código otimizado dos dois compiladores. Ainda não rodaram.

---

4. Como sabemos que a tradução está certa

O port não tem testes próprios. Os únicos testes são os do próprio WW3: os *regtests* (testes de regressão) que a NOAA mantém. Rodamos cada um duas vezes, com o Fortran e com o Kokkos, e comparamos os arquivos de saída com o `test.comp` da NOAA. Um arquivo diferente reprova a rotina (ADR-0010).

Testar em blocos. Os regtests são caros. Por isso o port anda em unidades grandes (por exemplo, toda a física ST4) e os testes rodam nos pontos de checagem: quando uma unidade acaba, quando muda o compilador e antes de qualquer medição de tempo. Entre os pontos de checagem, a paridade é argumentada comparando a representação intermediária do GCC (GIMPLE, com `-O3`) do Fortran e do C++ (ADR-0011).

Nenhum tempo sem paridade. Um número de velocidade só entra numa tabela depois que o regtest passou no mesmo commit e na mesma máquina. `scripts/results.py check` recusa o contrário.

Por que bit a bit é difícil. Exemplos reais deste repositório: o compilador junta `a*b + c` numa instrução só (FMA), que arredonda uma vez em vez de duas, e o resultado muda no último bit. Por isso o port compila com `-ffp-contract=off` e, na GPU, `--fmad=false`. Outro: o gfortran expande `x**11` de um jeito em `-O0` e de outro em `-O3` (v, `docs/log/2026-10-14-snl1-bitwise-af11.md`).

---

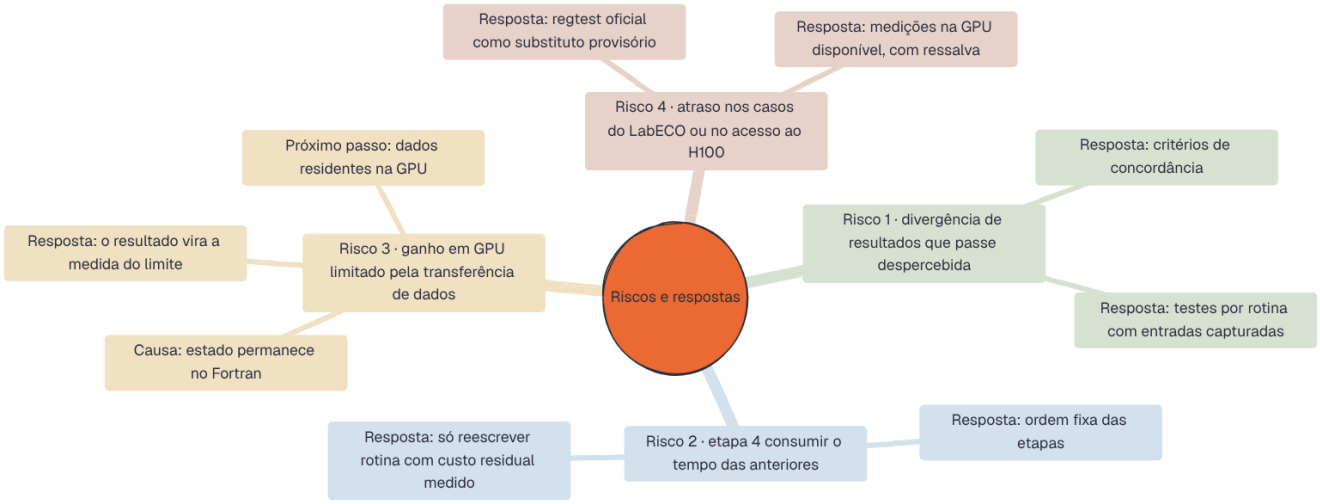
5. A H100: onde tradução e regtest se encontram

A pergunta do projeto é: o WW3 fica mais rápido na GPU **sem mudar a resposta**? A resposta só existe quando as duas metades estão na mesma máquina:

- **A tradução** diz que o código existe e compila para a GPU (hoje, 99,3 % do tempo).
- **O regtest** diz que esse código dá os mesmos bits que o Fortran (hoje, 48,4 %, e só na CPU).
- **O tempo** só vale quando o regtest passou ali mesmo.

Todo número de GPU que temos veio de uma RTX 4090. A issue h0ffmann/ww3-gpu#143 pede acesso à NVIDIA H100 do LabECO, na UFSC, e lista o que conferir lá: driver, CUDA para `sm_90`, Nix, contêineres, o escalonador e uso exclusivo da GPU para medir. Os regtests do WW3 rodando na H100, Fortran contra Kokkos, com a mediana de três rodadas, são a medição que mostra a paridade na velocidade da H100. Sem esse acesso, os números da H100 na proposta continuam sendo projeções (⚠).

Um risco já é conhecido: na GPU, funções como `expf` e `logf` da biblioteca da NVIDIA usam outros algoritmos que a `glibc`. Desligar o FMA é necessário, mas pode não bastar (⚠, `AGENTS.md`). Se uma rotina não conseguir dar os mesmos bits na GPU, o trabalho para e a decisão volta ao dono do projeto.



Os riscos do projeto e a resposta a cada um, da proposta. O "Risco 4" é o acesso à H100.

---

6. ADRs e WFIPs: como as decisões ficam escritas

Este projeto é feito em boa parte por agentes de código (Claude Code). Para que um agente possa ser cobrado, toda decisão e todo plano ficam escritos no repositório, com data e evidência.

ADR (*Architecture Decision Record*, registro de decisão) é um texto curto com uma escolha já feita: o contexto, a decisão, as alternativas e as consequências. Ele não agenda trabalho. Uma ADR aceita não é editada; uma nova a substitui. Hoje são dez (`docs/ADRs/`). As que mais importam para o port:

ADR	Decisão
ADR-0002	portar na ordem do tempo medido, a rotina mais cara primeiro
ADR-0010	o port não tem testes próprios; os regtests do WW3 são os únicos
ADR-0011	portar em unidades grandes e rodar os regtests nos pontos de checagem

ADR	Decisão
ADR-0001	não usar linguagem de prova formal; usar varreduras exaustivas, diff de GIMPLE e consultas SMT

WFIP (*Wave Forecaster Improvement Proposal*) é o desenho de uma mudança grande, escrito **antes** de ela ser feita. Cada WFIP aponta para um entregável da proposta do projeto (D1 a D6) e tem uma lista de "pronto" (*definition of done*) que conta quanto falta. Um script falha na CI quando o índice fica desatualizado. É parecido com um RFC ou um *design doc* de engenharia. Hoje são três (`docs/WFIPs/`):

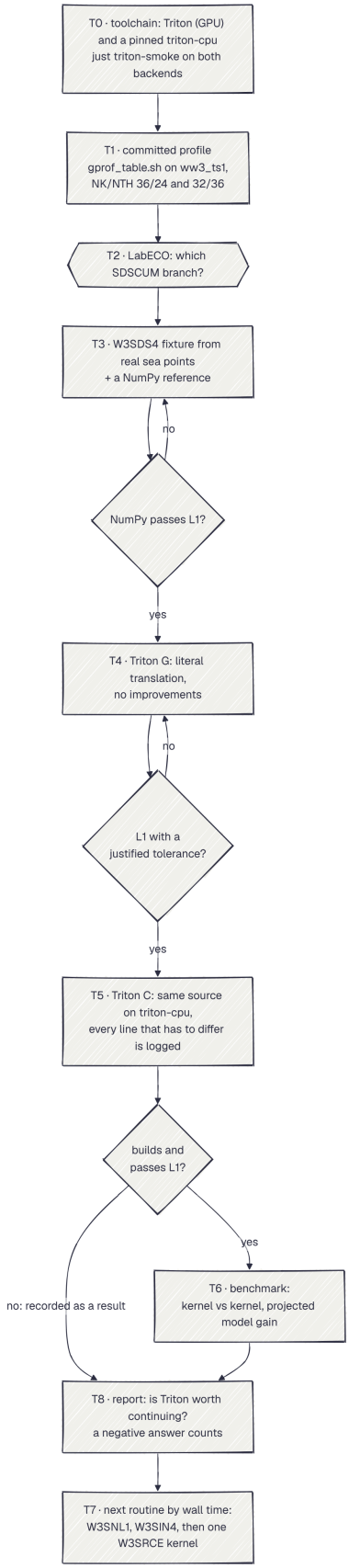
WFIP	O que desenha	Situação
WFIP-0001	rodar o WW3 com vento do WeatherNext 3 numa RTX 4090	adiada
WFIP-0002	o laboratório como livro, compilado do repositório	adiada
WFIP-0003	o DIA em Kokkos pela matriz de regtests do WW3, no fork	adiada, 7 de 7 itens prontos

As três foram adiadas em 2026-10-15 pela ADR-0011, que concentrou o trabalho no port.

---

7. O próximo passo e a outra frente

Depois da física ST4, a ordem segue o tempo medido (`kokkos/PORT_RANKING.md`): o `W3SRCE`, depois a propagação, depois a fase 2 (dados morando na GPU). Uma frente paralela testa se um kernel em Triton (a linguagem de kernels de GPU da OpenAI, escrita em Python, ⚠ `docs/GLOSSARY.md`) supera o Kokkos na parte mais cara do `W3SDS4` (`docs/W3SDS4_TRITON_PLANO_202610.pt.md`). A figura abaixo é o plano dessa frente. Ela foi feita antes da ADR-0010, por isso ainda fala em testes "L1", que saíram do projeto.



O plano do port do W3SDS4 para Triton, passo a passo, com as decisões em losangos.

8. Como citar

Cite o DOI que sempre abre a versão mais nova: 10.5281/zenodo.23221351. O registro no Zenodo é do tipo livro. Os dados para citação estão em `CITATION.cff` e no `README.md`.

Palavras difíceis, em uma frase cada

- **Backend:** o alvo para o qual o Kokkos compila o kernel (Serial, OpenMP, CUDA).
- **Bit a bit:** os dois resultados têm exatamente os mesmos bits, sem tolerância.
- **DIA:** o método aproximado que o WW3 usa para as trocas de energia entre ondas.
- **FMA:** uma instrução que faz `a*b + c` com um arredondamento só.
- **GIMPLE:** a representação intermediária do GCC, a mesma para Fortran e C++.
- **Regtest:** um caso de teste completo do WW3, com entrada e saída de referência.
- **Shim:** a camada fina que liga o Fortran ao C++.
- **Termo-fonte:** a parte do cálculo feita ponto a ponto, sem olhar os vizinhos.